

# SpringRestBenchmark

---

Datendurchsatz Schritt für Schritt optimieren  
Handbuch — Client ⇄ Spring Boot 4.1 ⇄ PostgreSQL

# Inhalt

- Ueberblick & Setup
- Optimierungs-Roadmap
- W0
- W1
- W2
- W3
- W4
- W5
- W6
- W7
- W8
- R0
- R1
- R2
- R3
- R4
- R5
- R6
- R7
- R8
- Aufgaben fuer Studenten

## SpringRestBenchmark

Ein **Lehrprojekt** rund um die Frage: *Wie holt man den maximalen Datendurchsatz zwischen React-Client, Spring-Boot-Server und PostgreSQL heraus?*

Das Projekt entwickelt die Antwort **in Stufen**. Jede Stufe demonstriert eine Optimierungstechnik, ist als eigener REST-Endpoint umgesetzt, wird im Browser gemessen und ausführlich dokumentiert. So wird die Verbesserungskurve unmittelbar sichtbar.

### Architektur

|         |                           |                                        |
|---------|---------------------------|----------------------------------------|
| client/ | React + Vite + TypeScript | – Benchmark-Runner, Recharts-Dashboard |
| server/ | Spring Boot 4.1 (Java 21) | – ein Endpoint je Optimierungsstufe    |
| infra/  | Docker Compose            | – PostgreSQL 16                        |
| docs/   | Markdown je Stufe         | – ausführliche Erklärungen             |

### Voraussetzungen

- **JDK 21** (unter `C:\Program Files\AdoptOpenJDK\jdk-21`)
- **Maven 4** (Aufruf über `ci 21` / `cit 21`) — ⚠️ `ci 11` funktioniert **nicht**, Spring Boot 4.1 verlangt mindestens JDK 17.
- **Docker** (für PostgreSQL)
- **Node.js 20+** und npm (für den Client)

### Schnellstart

```
# 1) Datenbank starten
docker compose -f infra/docker-compose.yml up -d

# 2) Server bauen und starten
cit 21                                # Build inkl. Tests (im Ordner server/)
java -jar server/target/benchmark-server-0.0.1-SNAPSHOT.jar

# 3) Client starten
npm install --prefix client
npm run dev --prefix client           # http://localhost:5173
```

Der Vite-Dev-Server leitet `/api/*` und `/actuator/*` per Proxy an den Server (Port 8080) weiter — es ist also keine CORS-Konfiguration nötig.

## Bedienung

1. Im Client die **Zeilenzahl** und optional die **Payload-Länge** wählen, dann **Seed** ausführen.
2. Eine einzelne **Stufe** (Button *Run*) oder einen ganzen Track auf einmal (► **Alle Write / Alle Read**) ausführen.
3. Das **Vergleichs-Dashboard** zeigt zwei Kurven — Write-Durchsatz (Zeilen/s) und Read-Nutzlast (Wire KB) — plus eine Detailtabelle (Serverzeit, TTFB, Bytes, Durchsatz).
4. Der **Daten-Viewer** unten rendert die Zeilen **virtualisiert** (nur sichtbare im DOM, flüssig auch bei 100 000+ Zeilen) und kann sie per **Streaming** inkrementell einfüllen.

## Mess-Prinzip

- Der Server setzt bei jeder Antwort den Header `Server-Timing: app;dur=<ms>` (reine Serverzeit). Schreibende Stufen liefern zusätzlich einen `BenchmarkResult` -Envelope mit Zeilenzahl und Serverzeit.
- Der Client misst clientseitig Gesamtzeit, Time-To-First-Byte und die tatsächlich übertragene Byte-Menge (aus dem Antwort-Stream).

## Tests

```
cit 21          # baut den Server inkl. aller Tests (JDK 21 + Maven 4)
```

- **Unit-Tests** für reine Logik (z. B. Durchsatz-Berechnung) — ohne Spring/DB, blitzschnell.
- **Integrationstests** gegen ein **echtes PostgreSQL** via **Testcontainers**: Die Basisklasse `AbstractPostgresIT` startet einmalig einen Container und verdrahtet die DataSource per `@ServiceConnection`; Flyway migriert das echte Schema.
- Konvention: Integrationstests heißen `*IntegrationTest` (laufen so in der Surefire- `test` -Phase), damit ein einziges `cit 21` **alle** Tests ausführt. Jede Optimierungsstufe bekommt mindestens einen Test.
- Voraussetzung: Docker muss laufen (Testcontainers zieht `postgres:16`).

## Lehrmaterial

- **Präsentation**: docs/slides.md — Marp-kompatible Folien (Export z. B. `marp docs/slides.md --pdf`).
- **Aufgaben für Studenten**: docs/AUFGABEN.md — gestufte Übungen (Vorhersagen, Experimente, Erweiterungen, Quiz).
- **Stufen-Doku**: docs/README.md mit `docs/W0-W8.md` und `docs/R0-R8.md`.

## Lasttests (Mehr-Nutzer)

Die Client-Messungen sind Einzel-Nutzer-Messungen. Für **nebenläufige** Last (viele gleichzeitige Nutzer, RPS, Latenz-Perzentile) gibt es [k6](#)-Skripte unter `infra/load/` — für Read- und Write-Last, auch ohne k6-Installation per Docker ausführbar.

## Optimierungs-Roadmap

Siehe docs/README.md für den vollständigen Stufenplan (Write-Track **W0-W8**, Read-Track **R0-R8**).

| Milestone | Inhalt                                                 | Status   |
|-----------|--------------------------------------------------------|----------|
| <b>M0</b> | Fundament: Infra, Server-Skelett, Mess-Harness, Client | ✓ fertig |
| <b>M1</b> | Baselines R0 + W0 (bewusst naiv)                       | ✓ fertig |
| <b>M2</b> | Write-Track W1-W8                                      | ✓ fertig |
| <b>M3</b> | Read-Track R1-R8                                       | ✓ fertig |
| <b>M4</b> | Client-Rendering + Vergleichs-Dashboard                | ✓ fertig |

# Optimierungs-Roadmap

Diese Übersicht listet alle geplanten Stufen. Zu jeder umgesetzten Stufe gehört eine eigene Markdown-Datei mit: *Konzept* · *Warum schneller* · *Welches Problem gelöst* · *Code-Walkthrough* · *gemessene Zahlen* · *Fallstricke/Trade-offs*.

## Write-Track — Ingest (Client → Server → DB)

| Stufe | Technik                                                               | Kernmittel                  | Doku    |
|-------|-----------------------------------------------------------------------|-----------------------------|---------|
| W0    | Baseline: 1 HTTP-Request pro Zeile, <code>save()</code> , Autocommit  | Spring Data JPA             | W0.md ✓ |
| W1    | Bulk-Payload: N Zeilen pro Request, weiter Einzel-INSERT (Autocommit) | Spring MVC / JPA            | W1.md ✓ |
| W2    | Alle Zeilen in EINER Transaktion (1 Commit)                           | <code>@Transactional</code> | W2.md ✓ |
| W3    | JDBC-Batch-INSERT ( <code>batchUpdate</code> )                        | Spring JDBC                 | W3.md ✓ |
| W4    | Batch + <code>rewriteBatchedInserts=true</code>                       | pgjdbc                      | W4.md ✓ |
| W5    | Chunk-orientierter Import                                             | Spring Batch                | W5.md ✓ |
| W6    | Bulk-Load per <code>COPY</code>                                       | pgjdbc CopyManager          | W6.md ✓ |
| W7    | Parallel-Ingest + Pool-Sizing                                         | Virtual Threads + HikariCP  | W7.md ✓ |
| W8    | Reactive Ingest mit Backpressure                                      | R2DBC (Reactor)             | W8.md ✓ |

Reihenfolge gegenüber dem ursprünglichen Plan leicht verfeinert, damit jede Stufe einen messbaren Effekt zeigt (Transaktionsgrenze vor Pool-Tuning; Pool-Sizing dort, wo unter Nebenläufigkeit wirksam — W7). Alle geplanten Techniken bleiben enthalten.

## Read-Track — Query (DB → Server → Client)

| Stufe | Technik                                                    | Kernmittel                | Doku    |
|-------|------------------------------------------------------------|---------------------------|---------|
| R0    | Baseline: <code>findAll()</code> → volle Entities als JSON | Spring Data JPA           | R0.md ✓ |
| R1    | DTO-Projektion (nur benötigte Spalten)                     | Spring JDBC               | R1.md ✓ |
| R2    | Pagination: Offset → Keyset/Seek                           | Spring JDBC               | R2.md ✓ |
| R3    | Server-seitiges Streaming (NDJSON, Cursor)                 | StreamingResponseBody     | R3.md ✓ |
| R4    | HTTP-Kompression (gzip)                                    | manuelle gzip-Kompression | R4.md ✓ |
| R5    | Caching                                                    | Spring Cache + Caffeine   | R5.md ✓ |
| R6    | Async/Parallel-Queries                                     | Virtual Threads           | R6.md ✓ |
| R7    | Binärformat statt JSON                                     | CBOR (Jackson 3)          | R7.md ✓ |
| R8    | Reactive End-to-End (NDJSON)                               | R2DBC (Reactor)           | R8.md ✓ |

## Client-Rendering (Bonus) ✓

Auch die Anzeige gehört zum Durchsatz „bis zur Wahrnehmung“:

- **Virtualisierte Liste** (TanStack Virtual): nur die sichtbaren Zeilen sind im DOM — flüssig auch bei 100 000+ Zeilen.
- **Streaming-Render**: Zeilen erscheinen inkrementell, während der NDJSON-Stream (R3) läuft.
- **Vergleichs-Dashboard**: Write-Durchsatz- und Read-Nutzlast-Kurven, „► Alle Write/Read ausführen“, Detailtabelle.

## Didaktischer Hinweis

Die Stufen sind bewusst so gebaut, dass der Durchsatz über den Verlauf **monoton steigt** — mit klar benannten Trade-offs (z. B. Keyset-Pagination verliert die Möglichkeit, direkt auf Seite  $n$  zu springen). Genau diese Trade-offs sind der Lernkern: Es gibt kein „schneller ist immer besser“, sondern „schneller unter welchen Bedingungen“.

# W0 — Write-Baseline: ein HTTP-Request pro Zeile

**Track:** Write (Client → Server → DB) **Ziel:** Die absichtlich schlechteste sinnvolle Implementierung. Sie liefert die Referenzzahl, gegen die alle folgenden Write-Stufen (W1–W8) antreten.

## Konzept

Der Client möchte  $N$  Zeilen in die Datenbank schreiben. In dieser Stufe tut er das auf die naheliegendste, aber ineffizienteste Weise:

1. Für **jede einzelne Zeile** wird ein **eigener HTTP-Request** abgesetzt.
2. Der Server nimmt genau eine Zeile entgegen und speichert sie mit `repository.save(...)`.
3. Es gibt **keine explizite Transaktionsklammer** — jeder `save()`-Aufruf läuft in seiner eigenen Transaktion und **committet einzeln**.

Damit summieren sich gleich **drei** Kostenblöcke, jeweils *pro Zeile*:

| Kostenblock     | Wo?             | Was passiert pro Zeile                   |
|-----------------|-----------------|------------------------------------------|
| HTTP-Round-Trip | Client ↔ Server | Verbindung, Header, Request/Response     |
| INSERT          | Server → DB     | ein einzelnes SQL-INSERT über den Pool   |
| COMMIT          | DB              | ein eigener Transaktions-Commit (fsync!) |

## Warum ist das langsam?

Der teuerste versteckte Posten ist oft der **Commit**: PostgreSQL muss bei jedem Commit das Write-Ahead-Log dauerhaft auf die Platte schreiben ( `fsync` ). Ein Commit pro Zeile bedeutet einen `fsync` pro Zeile. Dazu kommt der **HTTP-Overhead**: Selbst über `localhost` kostet jeder Request Zeit für Auf-/Abbau und Header-Verarbeitung — Zeit, in der *keine* Nutzdaten fließen.

Das Ergebnis: Der Durchsatz wird nicht von der Datenmenge bestimmt, sondern von der **Anzahl der Round-Trips und Commits**.

## Code-Walkthrough

**Server** — genau eine Zeile speichern:

- `controller/WriteController.java` → `w0SingleRow(...)`: nimmt einen einzelnen `MeasurementRequest` per `@RequestBody` entgegen.
- `service/WriteService.java` → `saveOne(...)`: ruft `repository.save(entity)` auf. Bewusst **ohne** `@Transactional`, damit jeder Aufruf einzeln committet.

**Client** — die Schleife ( `client/src/stages.ts`, Stufe `w0` ):

```
for (let i = 0; i < count; i++) {  
  const resp = await fetch('/api/write/w0', { method: 'POST', body });  
  await resp.arrayBuffer();  
}
```

Die Zeit wird über die **gesamte Schleife** clientseitig gemessen (Wanduhr), weil es keine einzelne „Serverzeit“ für den ganzen Vorgang gibt.

**Begrenzung:** W0 ist im Client auf **2000 Zeilen** gedeckelt ( `W0_MAX_ROWS` ). Ein HTTP-Request pro Zeile skaliert so schlecht, dass größere Zahlen den Unterricht nur aufhalten. Zum Vergleich zählt ohnehin der **Durchsatz** (Zeilen/s), nicht die absolute Zeilenzahl.

## Messung

Im Dashboard oben eine kleine Zeilenzahl wählen (z. B. 2000), dann **W0** ausführen. Beobachte:

- **Zeilen/s:** sehr niedrig — das ist die Referenz.
- **TTFB:** Zeit bis zur ersten Antwort (hier wenig aussagekräftig, da viele Requests).

Typische Größenordnung auf einem Entwickler-Notebook: **wenige Hundert bis wenige Tausend Zeilen/s**. Genau diese Zahl macht die späteren Sprünge eindrucksvoll.

## Fallstricke & Trade-offs

- **Nicht** so etwas in Produktion bauen. W0 existiert nur als Lehr-Referenz.
- Der `fsync`-pro-Commit-Effekt ist real, aber hardware-abhängig (SSD vs. HDD, WAL- Konfiguration). Deshalb sind **relative** Vergleiche aussagekräftiger als absolute.

## Ausblick

- **W1** behält die Einzel-INSERTs bei, sendet aber **alle Zeilen in einem Request** — isoliert damit den reinen HTTP-Overhead.
- **W3** bündelt die INSERTs zum **JDBC-Batch** — der eigentliche große Sprung.
- **W4** fasst alles in **eine Transaktion** — ein Commit statt *N*.

# W1 — Bulk-Payload, aber weiter Einzel-INSERT (Autocommit pro Zeile)

Track: Write · Vorgänger: W0 · Nachfolger: W2

## Konzept

W1 ändert gegenüber W0 **genau eine** Sache: Statt eines HTTP-Requests pro Zeile schickt der Client **alle Zeilen in einem einzigen Request** (ein JSON-Array). Der Server speichert sie aber weiterhin **einzel**n mit `repository.save(...)` — und **ohne** gemeinsame Transaktion, also mit **einem Commit pro Zeile**.

Damit isolieren wir sauber **einen** Kostenfaktor: den **HTTP-Overhead**.

|    | HTTP-Requests | Transaktionen/Commits | INSERTs |
|----|---------------|-----------------------|---------|
| W0 | N             | N                     | N       |
| W1 | 1             | N                     | N       |

## Was die Messung zeigt

Beispiel-Lauf (2000 Zeilen, lokales Notebook):

| Stufe | Zeilen | Server ms | Zeilen/s |
|-------|--------|-----------|----------|
| W1    | 2000   | 17916     | 112      |

**Die entscheidende Erkenntnis:** Der HTTP-Overhead war *nicht* der Hauptkostentreiber. W1 ist gegenüber W0 kaum schneller, weil die **N Commits** (je ein `fsync` aufs Write-Ahead-Log) den Durchsatz bestimmen — und die sind in W1 noch genauso da wie in W0.

Das ist die eigentliche Lehre dieser Stufe: Man optimiert leicht die *falsche* Sache. „Weniger HTTP“ klingt gut, bringt hier aber fast nichts. Erst W2 fasst die Commits zusammen — und dann springt der Durchsatz.

## Code-Walkthrough

- `service/WriteService.java` → `w1BulkAutocommit(List<MeasurementRequest>)` : Schleife mit `repository.save(...)`. **Kein** `@Transactional` → jeder `save()` committet einzeln.
- `controller/WriteController.java` → `w1(...)` : leert die Tabelle (nicht gemessen), misst dann die reine Insert-Zeit und liefert sie im `BenchmarkResult`-Envelope zurück.
- Client (`client/src/stages.ts`, `runBulkWrite`): baut N Zeilen, schickt sie als **ein** Array, liest die Serverzeit aus dem Envelope. W1 ist auf **20 000 Zeilen** gedeckelt.

## Fallstricke & Trade-offs

- Auch hier gilt: nur als Lehr-Stufe gedacht. Per-Zeile-Commit gehört nicht in Produktion.
- Der Effekt (bzw. Nicht-Effekt) hängt stark von der Platte/WAL-Konfiguration ab — **relative** Vergleiche sind aussagekräftiger als absolute Zahlen.

## Ausblick

- **W2** legt alle Zeilen in **eine** Transaktion → nur noch ein Commit.

## W2 — Alle Zeilen in EINER Transaktion

Track: Write · Vorgänger: W1 · Nachfolger: W3

### Konzept

W2 ändert gegenüber W1 wieder **genau eine** Sache: Die Insert-Schleife läuft jetzt in **einer gemeinsamen Transaktion** (`@Transactional`). Statt N Commits gibt es **einen**.

|    | HTTP-Requests | Transaktionen/Commits | INSERTs |
|----|---------------|-----------------------|---------|
| W1 | 1             | N                     | N       |
| W2 | 1             | 1                     | N       |

### Was die Messung zeigt

Beispiel-Lauf (2000 Zeilen, lokales Notebook):

| Stufe | Zeilen | Server ms | Zeilen/s | Faktor ggü. W1 |
|-------|--------|-----------|----------|----------------|
| W1    | 2000   | 17916     | 112      | —              |
| W2    | 2000   | 4362      | 459      | ≈ 4×           |

**Erkenntnis:** Der große Hebel war das Zusammenfassen der **Commits**, nicht das Bündeln der INSERTs. Allein „ein Commit statt N“ bringt hier den Faktor ~4.

### Warum ist W2 *absolut* immer noch langsam?

459 Zeilen/s sind besser, aber weit weg von den späteren Stufen. Grund ist ein bekanntes **JPA-Anti-Pattern**: In der einen Transaktion sammelt Hibernate **alle** Entities im Persistence-Context. Weil die Entity `GenerationType.IDENTITY` nutzt, führt jeder `save()` sofort ein INSERT aus — und der wachsende Persistence-Context macht die Dirty-Checks zunehmend teurer (Richtung  $O(n^2)$ ), wenn man nicht periodisch `flush()` / `clear()` aufruft.

Genau deshalb wechseln die nächsten Stufen von JPA auf **rohes JDBC**: Bei IDENTITY-Schlüsseln kann Hibernate INSERTs ohnehin nicht als JDBC-Batch bündeln.

### Code-Walkthrough

- `service/WriteService.java` → `w2SingleTransaction(...)` : identische Schleife wie W1, aber mit `@Transactional`. Der externe Aufruf aus dem Controller aktiviert den Proxy, sodass die Transaktion die ganze Schleife umschließt und erst am Methodenende committet.
- `controller/WriteController.java` → `w2(...)` : misst die Zeit inklusive Commit.

### Fallstricke & Trade-offs

- Eine große Transaktion hält Sperren und Ressourcen länger. Bei sehr vielen Zeilen kann ein **Chunk-Commit** (periodisches Committen, siehe W5 / Spring Batch) besser sein als „alles oder nichts“.
- Der wachsende Persistence-Context ist ein realer Speicher-/CPU-Kostenpunkt — ein Grund, für Massen-Insert nicht JPA, sondern JDBC/COPY zu nutzen.

## Ausblick

- **W3** ersetzt die N Einzel-INSERTs durch **ein** `JdbcTemplate.batchUpdate` → weniger Round-Trips zur DB.

## W3 — JDBC-Batch-INSERT

Track: Write · Vorgänger: W2 · Nachfolger: W4

### Konzept

Bis W2 wurde jede Zeile mit einem eigenen INSERT an die Datenbank geschickt — also  $N$  Hin- und Rückwege (Round-Trips) über die Verbindung. W3 bündelt sie: Mit `JdbcTemplate.batchUpdate(...)` werden die Zeilen in **Chunks** (hier je 1000) als **ein JDBC-Batch** übertragen. Ein Round-Trip transportiert 1000 INSERTs.

|    | Round-Trips zur DB               | Technik                                      |
|----|----------------------------------|----------------------------------------------|
| W2 | $N$ (ein INSERT pro Zeile)       | JPA <code>save()</code> in einer Transaktion |
| W3 | $N / 1000$ (ein Batch pro Chunk) | rohes JDBC <code>batchUpdate</code>          |

### Warum rohes JDBC statt JPA?

Die Entity nutzt `GenerationType.IDENTITY`. Damit muss Hibernate nach **jedem** INSERT sofort die generierte `id` lesen — und kann INSERTs deshalb **nicht** zu einem JDBC-Batch zusammenfassen. Wer mit JPA + IDENTITY „batchen“ will, bekommt trotzdem  $N$  Einzel-INSERTs. Deshalb umgehen wir Hibernate hier bewusst und sprechen die DB direkt über `JdbcTemplate` an.

### Was die Messung zeigt

| Stufe | Zeilen | Server ms | Zeilen/s | Faktor ggü. W2 |
|-------|--------|-----------|----------|----------------|
| W2    | 5000   | 10035     | 498      | —              |
| W3    | 100000 | 10339     | 9 672    | ≈ 19×          |

Der Sprung ist enorm: Das Bündeln der Round-Trips ist der bislang größte Hebel. W3 verarbeitet in *derselben* Zeit rund das 20-fache an Zeilen wie W2.

### Code-Walkthrough

- `service/WriteService.java` → `w3JdbcBatch(...)` → `batchInsert(jdbcTemplate, rows)`: nutzt die Überladung `batchUpdate(sql, rows, BATCH_SIZE, setter)`, die die Liste automatisch in Chunks von `BATCH_SIZE` zerlegt und pro Chunk einen Batch schickt.
- Der `ParameterizedPreparedStatementSetter` setzt die 12 Spaltenwerte je Zeile.
- `controller/WriteController.java` → `w3(...)`: misst die reine Batch-Zeit.

### Fallstricke & Trade-offs

- Chunk-Größe** ist ein Tuning-Parameter: zu klein → mehr Round-Trips; zu groß → mehr Treiber-/DB-Speicher pro Batch. 500–2000 ist ein üblicher Startbereich.
- Der Batch umgeht die JPA-Ebene — für reine Massen-Inserts ist das gewollt, man verliert aber Entity-Lifecycle, Dirty-Checking etc. (hier irrelevant).

### Ausblick

- W4 aktiviert `rewriteBatchedInserts=true`: pgjdbc formt den Batch in **Multi-Row-INSERTs** um und wird dadurch nochmals deutlich schneller.

## W4 — JDBC-Batch mit `rewriteBatchedInserts=true`

Track: Write · Vorgänger: W3 · Nachfolger: W5

### Konzept

W4 ändert gegenüber W3 **nur eine Verbindungseinstellung**: den pgjdbc-Parameter `rewriteBatchedInserts=true`. Damit erkennt der Treiber, dass ein Batch aus lauter gleichartigen INSERTs besteht, und **schreibt ihn in ein Multi-Row-INSERT um**:

```
-- statt 1000x "INSERT INTO measurements (...) VALUES (?, ...)"
INSERT INTO measurements (...) VALUES (?, ...), (?, ...), (?, ...), ... -- 1000 Tupel
```

Ein einziges Statement statt 1000 — das spart Parsing- und Protokoll-Overhead auf der DB-Seite erheblich.

### Was die Messung zeigt

| Stufe     | Zeilen | Server ms | Zeilen/s      | Faktor ggü. W3 |
|-----------|--------|-----------|---------------|----------------|
| W3        | 100000 | 10339     | 9 672         | —              |
| <b>W4</b> | 100000 | 2664      | <b>37 541</b> | <b>≈ 3,9×</b>  |

Gegenüber der Transaktions-Stufe W2 (498 Zeilen/s) ist W4 rund **75× schneller** — allein durch bessere Insert-Technik, ohne Parallelität.

### Code-Walkthrough — die zweite Verbindung

`rewriteBatchedInserts` ist eine **Verbindungseigenschaft** und muss beim Aufbau der Connection gesetzt sein; man kann sie nicht pro Statement umschalten. Damit W3 und W4 denselben Code, aber unterschiedliche Verbindungen nutzen, gibt es zwei `JdbcTemplate`s:

- `config/BatchDataSourceConfig.java` :
  - `jdbcTemplate (@Primary)` → Boots normale DataSource (ohne den Parameter) → W3.
  - `rewriteJdbcTemplate` → zweite Verbindung mit `rewriteBatchedInserts=true` → W4. Die zweite Verbindung wird **aus der primären DataSource abgeleitet** (gleiche URL/Zugangsdaten, nur um den Parameter ergänzt). So funktioniert sie unverändert auch in den Integrationstests, wo die DataSource per `@ServiceConnection` auf einen Testcontainer zeigt.
- `service/WriteService.java` → `w4JdbcBatchRewrite(...)` ruft dieselbe `batchInsert(...)` auf, nur mit dem `rewriteJdbcTemplate`.

Bewusste Vereinfachung: Die zweite DataSource wird **nicht** als Spring-Bean exponiert, damit Boots JPA-/Flyway-Autokonfiguration (die eine eindeutige DataSource erwartet) nicht gestört wird.

### Fallstricke & Trade-offs

- `rewriteBatchedInserts` wirkt nur bei **einfachen** `INSERT ... VALUES`-Statements (kein `ON CONFLICT`-Sonderfall etc.). Für unseren Fall ideal.
- Sehr große umgeschriebene Statements können Grenzen für Parameteranzahl/Statementgröße treffen — die Chunk-Größe (1000) hält das im Rahmen.

## Ausblick

- **W5** zeigt dieselbe Batch-Idee mit **Spring Batch** (chunk-orientiert, restartfähig) — wenn nicht nur roher Durchsatz, sondern auch Robustheit zählt.

## W5 — Spring Batch (chunk-orientierter Import)

Track: Write · Vorgänger: W4 · Nachfolger: W6

### Konzept

Bis hierher haben wir Durchsatz durch bessere *Insert-Technik* gewonnen. W5 wechselt die Perspektive: Es nutzt **Spring Batch** — ein Framework für robuste Massenverarbeitung. Die Zeilen laufen durch einen echten **Job** mit einem **chunk-orientierten Step**:

```
ListItemReader → (optional Processor) → JdbcBatchItemWriter
                \_____ Chunk (1000) _____/
                  nach jedem Chunk: Commit
```

Nach jedem Chunk committet Spring Batch und schreibt den Fortschritt in seine **Metadatentabellen** ( `BATCH_JOB_EXECUTION` , `BATCH_STEP_EXECUTION` , ...).

### Was die Messung zeigt

| Stufe                    | Zeilen | Server ms | Zeilen/s       |
|--------------------------|--------|-----------|----------------|
| W3 (JDBC-Batch)          | 100000 | ~6600     | ~15 000        |
| <b>W5 (Spring Batch)</b> | 100000 | ~4500     | <b>~22 000</b> |
| W6 (COPY)                | 100000 | ~1300     | ~76 000        |

**Wichtig:** W5 liegt in derselben Größenordnung wie W3/W4 — es benutzt darunter dieselbe JDBC-Batch-Technik. Der Gewinn von W5 ist **nicht** höherer Durchsatz, sondern:

- **Chunk-Commit** statt „alles oder nichts“ (schonender bei sehr großen Mengen),
- **Restart/Wiederaufsetzen** nach Abbruch (der JobRepository weiß, wo es weiterging),
- **Skip-/Retry-Policies**, Listener, **Monitoring** über die Batch-Metadaten.

Kernbotschaft: Ein Batch-*Framework* wählt man für **Robustheit und Betreibbarkeit**, nicht für den letzten Prozentpunkt Geschwindigkeit. Wer nur rohen Durchsatz will, nimmt W6.

### Code-Walkthrough

- `config/BatchJobConfig.java` → `measurementItemWriter(...)`: ein wiederverwendbarer `JdbcBatchItemWriter`, der Chunks als JDBC-Batch schreibt (positionsbasierter `ItemPreparedStatementSetter`, weil das Record keine Bean-Getter auf Spaltennamen hat).
- `service/WriteService.java` → `w5SpringBatch(...)`: baut **pro Request** Step + Job ( `StepBuilder` / `JobBuilder` ), weil die Daten aus dem Request kommen. Der Reader ist ein `ListItemReader` über die gelieferten Zeilen. Der Lauf wird über den `JobLauncher` mit **eindeutigen Job-Parametern** gestartet (sonst hielte Spring Batch ihn für einen bereits gelaufenen Job).
- `db/migration/V2__spring_batch_schema.sql`: das Batch-Metadatenschema legt **Flyway** an (nicht Spring Batch selbst; `spring.batch.jdbc.initialize-schema=never`).

## Fallstricke & Trade-offs

- **API-Evolution:** Spring Batch 6 markiert `JobLauncher` und das klassische Chunk-Modell als *deprecated* (Nachfolger in 7.0: `JobOperator` bzw. ein neues Verarbeitungsmodell). Wir nutzen bewusst die aktuell von Spring Boot 4.1 autokonfigurierte, dokumentierte Variante — ein gutes Beispiel dafür, dass Frameworks sich weiterentwickeln.
- **Job pro Request** ist für ein ad-hoc-Benchmark ungewöhnlich; im Alltag definiert man Jobs als Beans und startet sie geplant. Hier stammen die Daten aber aus dem HTTP-Request.
- Die Metadatentabellen wachsen mit jedem Lauf — in echten Systemen räumt man sie auf.

## Ausblick

- **W6** zeigt den reinen Geschwindigkeits-Gegenpol: Postgres **COPY**.
- **W7** bringt **Parallelität** (Virtual Threads) und macht die **Pool-Größe** wichtig.

## W6 — Postgres COPY (nativer Bulk-Load)

Track: Write · Vorgänger: W5 / W4 · Nachfolger: W7

### Konzept

Alle bisherigen Stufen benutzen `INSERT` — und selbst der schnellste `INSERT`-Weg (W4) muss jede Zeile durch den regulären Anfrage-Pfad der Datenbank schleusen (Parsen, Planen, Ausführen). PostgreSQL hat für Massendaten ein **eigenes Protokoll**: `COPY ... FROM STDIN`. Es überträgt die Zeilen als Datenstrom und schreibt sie auf dem kürzesten Weg in die Tabelle. Das ist der **schnellste** Weg, große Datenmengen zu laden.

Wir sprechen `COPY` direkt über den `pgjdbc-CopyManager` an.

### Was die Messung zeigt

| Stufe | Zeilen | Server ms | Zeilen/s | Faktor ggü. W4 |
|-------|--------|-----------|----------|----------------|
| W4    | 100000 | 2180      | 45 875   | —              |
| W6    | 100000 | 1211      | 82 590   | ≈ 1,8×         |

Gegenüber der Transaktions-Stufe W2 (498 Zeilen/s) ist `COPY` rund **166×** schneller.

### Code-Walkthrough

- `service/WriteService.java` → `w6Copy(...)` :
  - holt eine `Connection` aus der `DataSource`,
  - entpackt sie zu `PGConnection` (`connection.unwrap(PGConnection.class)`),
  - baut mit `toCsv(...)` einen CSV-Text und übergibt ihn an `copyManager.copyIn("COPY measurements (...) FROM STDIN WITH (FORMAT csv)", reader)`.
- `toCsv(...)` : erzeugt pro Zeile eine CSV-Zeile. Unsere Daten enthalten keine Sonderzeichen (Kategorie aus fester Liste, Payload nur Kleinbuchstaben), daher ist **kein CSV-Escaping** nötig. Ein **leeres** Feld bedeutet im CSV-Modus `NULL` — genau richtig für ein fehlendes `payload` (dafür gibt es einen eigenen Test).

### Fallstricke & Trade-offs

- Escaping**: Bei echten Daten mit Kommas, Anführungszeichen oder Zeilenumbrüchen **muss** man CSV korrekt escapen (oder das Text-/Binärformat von `COPY` nutzen). Unsere Vereinfachung gilt nur, weil die Testdaten kontrolliert sind.
- NULL vs. leerer String**: Im CSV-Modus ist ein *unquoted* leeres Feld `NULL`, ein *quoted* `""` der leere String. Diese Feinheit ist eine typische Fehlerquelle.
- `COPY` umgeht Trigger-freundliche Einzelverarbeitung — es ist für reinen Bulk-Load gedacht.

### Ausblick

- W7** verlässt die „eine Verbindung, seriell“-Welt: Es verteilt den Ingest über **mehrere Verbindungen parallel** (Virtual Threads) — und hier wird endlich die **Größe des Connection-Pools** (HikariCP) zum entscheidenden Faktor.

# W7 — Parallel-Ingest über Virtual Threads (+ Pool-Sizing)

Track: Write · Vorgänger: W6 · Nachfolger: W8

## Konzept

Alle bisherigen Stufen arbeiten **seriell** — ein Thread, eine Verbindung. W7 verteilt den Ingest auf **mehrere Verbindungen gleichzeitig**: Die Zeilen werden in Partitionen aufgeteilt, jede Partition läuft als eigener Task (JDBC-Batch mit `rewriteBatchedInserts=true`), und alle Tasks laufen **parallel** auf einem **Virtual-Thread-pro-Task-Executor**.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (var part : partitions) futures.add(executor.submit(() -> batchInsert(part)));
    ...
}
```

**Virtual Threads** (Java 21) machen Nebenläufigkeit billig: Man kann bedenkenlos viele Tasks starten, ohne einen knappen Plattform-Thread-Pool zu erschöpfen.

## Die eigentliche Lektion: Pool-Größe ↔ Parallelität

Parallelität hilft nur, wenn die Datenbank **genug Verbindungen** bereitstellt. Deshalb nutzt W7 einen **eigenen HikariCP-Pool mit 16 Verbindungen** ( $\geq$  Parallelitätsgrad 8). Mit dem Standard-Pool (10) oder gar einem kleinen Pool würden die Tasks um Verbindungen konkurrieren und sich **serialisieren** — der Parallelitätsvorteil verpufft.

Genau hier wird das Connection-Pool-Sizing, das in einer seriellen Welt (W2–W6) kaum eine Rolle spielt, zum entscheidenden Faktor.

## Was die Messung zeigt

| Stufe                          | Zeilen | Server ms | Zeilen/s       |
|--------------------------------|--------|-----------|----------------|
| W4 (seriell, Batch+rewrite)    | 100000 | ~2250     | ~44 000        |
| <b>W7 (parallel, VThreads)</b> | 100000 | ~1190     | <b>~84 000</b> |
| W6 (COPY, seriell)             | 100000 | ~975      | ~102 000       |

W7 verdoppelt gegenüber dem seriellen W4 grob den Durchsatz. Ob COPY (W6) noch überholt wird, hängt stark von Kernen, Platte und WAL-Konfiguration ab — auf write-lastigen Systemen ist COPY oft schwer zu schlagen, aber W7 kommt nah heran.

## Fallstricke & Trade-offs

- **Pool zu klein → kein Nutzen.** Der wichtigste Stolperstein; deshalb der dedizierte Pool.
- Mehr Parallelität ist nicht beliebig gut: Zu viele gleichzeitige Schreiber erzeugen **Contention** (Locks, WAL, I/O). Der optimale Grad ist ein Tuning-Thema.
- Reihenfolge/Transaktionalität über Partitionen hinweg geht verloren (jede Partition committet für sich) — für reinen Bulk-Insert akzeptabel.

## Ausblick

- **W8** wechselt das Programmiermodell komplett: **reaktiv/non-blocking** über R2DBC — und zeigt, dass „reaktiv“ nicht automatisch „schneller beim Bulk-Insert“ heißt.

## W8 — Reaktiver Ingest über R2DBC

Track: Write · Vorgänger: W7 · Abschluss des Write-Tracks

### Konzept

W8 wechselt das **Programmiermodell**: weg vom blockierenden JDBC, hin zu **R2DBC** (Reactive Relational Database Connectivity). Die Zeilen laufen als reaktiver **Reactor-Strom**:

```
Flux.fromIterable(rows)
    .buffer(BATCH_SIZE)                // in Chunks gruppieren
    .flatMap(this::insertChunk, CONCURRENCY) // begrenzt gleichzeitig -> Backpressure
    .reduce(0L, Long::sum)
    .block();                          // Ergebnis an der MVC-Grenze einsammeln
```

Der Insert-Pfad selbst ist vollständig **non-blocking**; die begrenzte Nebenläufigkeit ( `CONCURRENCY` ) ist die **Backpressure**: Der Verbraucher (DB) gibt das Tempo vor, der Produzent überrennt ihn nicht.

### Was die Messung zeigt — die ehrliche Lektion

| Stufe                     | Zeilen | Server ms | Zeilen/s    |
|---------------------------|--------|-----------|-------------|
| W2 (JPA, 1 Transaktion)   | 5000   | ~10000    | ~500        |
| <b>W8 (R2DBC reaktiv)</b> | 20000  | ~21900    | <b>~900</b> |
| W3 (JDBC-Batch)           | 100000 | ~6600     | ~15 000     |
| W6 (COPY)                 | 100000 | ~975      | ~100 000    |

**W8 ist beim reinen Bulk-Insert langsam** — hier sogar mit am langsamsten. Das ist kein Fehler, sondern die Kernbotschaft:

**Reaktiv ist kein Turbo-Knopf für Durchsatz.** R2DBC/Reactive glänzt bei **hoher Nebenläufigkeit** und **I/O-Wartezeiten** (viele gleichzeitige, teils langsame Clients, Streaming, effiziente Thread-Nutzung) — nicht beim stumpfen Massen-Insert. Für reinen Bulk-Load bleibt **COPY** (W6) das Mittel der Wahl.

Deshalb ist W8 im Client auf **20 000 Zeilen** begrenzt (sonst dauert der Lauf im Browser zu lange).

### Code-Walkthrough

- `service/WriteService.java` → `w8ReactiveIngest(...)`, `insertChunkReactive(...)`, `bindRowReactive(...)`: reaktiver Chunk-Insert über die R2DBC- `Statement` -API (Platzhalter `$1..$12`, `statement.add()` je Zeile).
- Wichtige Architektur-Entscheidung** ( `buildConnectionFactory` im `WriteService`): Die R2DBC- `ConnectionFactory` ist bewusst **keine Spring-Bean**, sondern ein privates Feld, abgeleitet aus der JDBC-`DataSource`-URL. Grund: Spring Boots `DataSourceAutoConfiguration` ist `@ConditionalOnMissingBean(ConnectionFactory)` — eine `ConnectionFactory` -**Bean** würde die JDBC-`DataSource`-Autokonfiguration komplett abschalten (Reactive-First-Heuristik) und damit JPA/Flyway lahmlegen. Als privates Feld umgehen wir das und lassen JDBC und R2DBC friedlich koexistieren.

### Fallstricke & Trade-offs

- `block()` **an der Grenze**: Dieser Endpoint lebt in der (blockierenden) MVC-Welt, daher wird das reaktive Ergebnis am Ende eingesammelt. In einer echten WebFlux-Anwendung bliebe die Kette bis zum HTTP-Response reaktiv (das zeigt

der Read-Track mit SSE, R8).

- **Kein `rewriteBatchedInserts`** : Der R2DBC-Postgres-Treiber bündelt Batches nicht in Multi-Row-INSERTs — ein Grund für den geringeren Durchsatz gegenüber W4.
- Mischbetrieb JDBC + R2DBC will bewusst konfiguriert sein (siehe Architektur-Hinweis oben).

## Fazit des Write-Tracks

Von W0 (1 Request/Zeile) bis W6 (COPY) steigt der Durchsatz um **Größenordnungen** — der größte Hebel war das **Bündeln** (erst Commits, dann Round-Trips), gefolgt vom **richtigen DB-Werkzeug** (COPY). W7 zeigt **Parallelität**, W8 ein **anderes Programmiermodell**. Die wichtigste Erkenntnis: Es gibt kein „schneller ist immer besser“ — jede Technik hat ihren Kontext, ihre Trade-offs und ihren richtigen Einsatzort.

# R0 — Read-Baseline: die komplette Tabelle als ein JSON-Array

**Track:** Read (DB → Server → Client) **Ziel:** Die absichtlich schlechteste sinnvolle Leseimplementierung. Referenz für die Read-Stufen R1–R8.

## Konzept

Der Client möchte Messdaten anzeigen. In dieser Stufe holt der Server sie auf die naivste Weise:

1. `repository.findAll()` lädt **alle** Zeilen als **vollständige Entities** in eine `List` im Server-Speicher.
2. Diese Liste wird **komplett** in **ein großes JSON-Array** serialisiert.
3. Erst wenn das gesamte Array fertig ist, geht die Antwort an den Client.

## Warum ist das schlecht?

Gleich drei Schwächen greifen zusammen:

| Schwäche                                                                | Folge                                                                                            | Wird angegriffen von            |
|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------|
| <b>Alles auf einmal</b> in den RAM                                      | Speicher wächst linear mit der Tabelle; bei Millionen Zeilen droht <code>OutOfMemoryError</code> | R2 (Pagination), R3 (Streaming) |
| <b>Vollständige Entities</b> (alle Spalten inkl. <code>payload</code> ) | unnötig große Nutzlast, auch wenn der Client nur wenige Felder braucht                           | R1 (DTO-Projektion)             |
| <b>Erst fertig serialisieren, dann senden</b>                           | hohe <b>Time-To-First-Byte</b> ; der Client sieht lange gar nichts                               | R3 (Streaming), R8 (Reactive)   |

Zusätzlich muss der Client das **gesamte** JSON parsen, bevor er etwas anzeigen kann — auch das kostet spürbar Zeit und Speicher.

## Code-Walkthrough

**Server:**

- `service/ReadService.java` → `findAllNaive()`: `return repository.findAll();`, annotiert mit `@Transactional(readOnly = true)`.
- `controller/ReadController.java` → `r0FindAll()`: gibt die `List<Measurement>` **direkt** zurück. Jackson serialisiert die **Entities selbst** — bewusst das Anti-Pattern, das R1 mit einem schlanken DTO auflöst.

**Client** ( `client/src/stages.ts`, Stufe `r0`):

```
const m = await measure('/api/read/r0', { method: 'GET' });
const rows = (JSON.parse(m.bodyText) as unknown[]).length;
```

Das Mess-Harness zählt die **übertragenen Bytes** direkt aus dem Antwort-Stream und liest die **Serverzeit** aus dem `Server-Timing`-Header.

## Messung

Voraussetzung: vorher **Seed** ausführen (z. B. 100 000 Zeilen). Dann **R0** starten. Beobachte in der Tabelle:

- **KB / MB**: die volle Nutzlast auf der Leitung — inkl. aller Spalten und `payload`.
- **TTFB ≈ Gesamtzeit**: typisch für „erst alles fertig, dann senden“.

- **Server ms:** Zeit zum Laden + Serialisieren im Server.

Merke dir diese Zahlen — R1 (Projektion) senkt zuerst die **Bytes**, R3 (Streaming) senkt vor allem die **TTFB**.

## Fallstricke & Trade-offs

- R0 **funktioniert** für kleine Tabellen völlig problemlos — genau das macht es gefährlich: Es fällt erst bei großem Datenbestand um. Für die Demo daher ruhig mit vielen Zeilen (und `payloadLength > 0`) seeden, damit der Effekt sichtbar wird.
- Entities direkt zu serialisieren koppelt zudem die API an das DB-Schema — ein Wartungsproblem, das R1 nebenbei mitlöst.

## Ausblick

- **R1** liefert statt der ganzen Entity ein **DTO** mit nur den benötigten Spalten → weniger Bytes.
- **R2** holt die Daten **seitenweise** (Keyset) statt alles auf einmal.
- **R3 streamt** die Zeilen (NDJSON) → niedrige TTFB, konstanter Speicher.

# R1 — DTO-Projektion

Track: Read · Vorgänger: R0 · Nachfolger: R2

## Konzept

R0 serialisiert die **vollständige Entity** — alle Spalten, inklusive der acht Messwerte `v1..v8` und des potenziell großen `payload`. R1 liefert stattdessen ein **schlankes DTO** mit nur den Feldern, die der Client wirklich braucht: `id`, `ts`, `sensorId`, `category`, `v1`.

Statt JPA-Entities zu materialisieren, selektiert R1 direkt per JDBC nur die benötigten Spalten — das spart auch serverseitig Arbeit.

## Was die Messung zeigt

100 000 Zeilen, mit `payloadLength=64` geseedet:

| Stufe                  | Nutzlast | Faktor               | Zeit  |
|------------------------|----------|----------------------|-------|
| R0 (volle Entity)      | 35,4 MB  | —                    | 8,8 s |
| <b>R1 (Projektion)</b> | 10,7 MB  | <b>~3,3× kleiner</b> | 1,3 s |

Die Projektion **drittelt** die Nutzlast und ist hier  $\sim 7\times$  schneller — allein dadurch, dass die schweren Felder (`v2-v8`, `payload`) gar nicht erst übertragen werden. Mit größerem `payload` wird der Unterschied noch drastischer.

## Code-Walkthrough

- `dto/MeasurementDto.java` : das schlanke Record (`id`, `ts`, `sensorId`, `category`, `v1`).
- `service/ReadService.java` → `projection()` : `jdbcTemplate.query(...)` mit einem `RowMapper`, der genau diese Spalten liest — **kein** Laden voller Entities.
- `controller/ReadController.java` → `r1Projection()` : gibt die DTO-Liste als JSON zurück.

## Fallstricke & Trade-offs

- Ein DTO pro Sichtweise:** Verschiedene Clients brauchen verschiedene Felder — zu viele DTOs werden unübersichtlich. Alternativen: GraphQL, Sparse Fieldsets, projizierende Query-Sprachen.
- Die Projektion entkoppelt die API vom DB-Schema (gut), erfordert aber Pflege beim Mapping.
- Es wird weiterhin **alles auf einmal** geladen und gesendet — das lösen erst R2 (Pagination) und R3 (Streaming).

## Ausblick

- R2** holt die Daten **seitenweise** statt alles auf einmal (Offset vs. Keyset).
- R4** komprimiert die (bereits schlanke) Projektion zusätzlich per gzip.

## R2 — Pagination: Offset vs. Keyset

Track: Read · Vorgänger: R1 · Nachfolger: R3

### Konzept

R0/R1 laden **alles auf einmal**. R2 holt die Daten stattdessen **seitenweise**. Dabei gibt es zwei grundverschiedene Techniken:

**Offset-Pagination** ( `ORDER BY id OFFSET ? LIMIT ?` ): Die DB sortiert, **überspringt** `OFFSET` Zeilen und liefert die nächsten `LIMIT`. Das Überspringen bedeutet: Für Seite  $n$  muss die DB  $n \times \text{Seitengröße}$  Zeilen durchlaufen — tiefe Seiten werden **immer langsamer**.

**Keyset-Pagination / Seek** ( `WHERE id > ? ORDER BY id LIMIT ?` ): Statt zu überspringen, **springt** die DB per Index direkt hinter den zuletzt gesehenen Schlüssel. Jede Seite kostet gleich viel — **unabhängig von der Tiefe**.

### Was die Messung zeigt

100 000 Zeilen, eine **tiefe** Seite (Position ~95 000, Größe 5000):

| Technik                                     | Zeit für die tiefe Seite |
|---------------------------------------------|--------------------------|
| Offset ( <code>OFFSET 95000</code> )        | ~0,208 s                 |
| Keyset ( <code>WHERE id &gt; 95000</code> ) | ~0,062 s                 |

Keyset ist hier ~3,4× schneller — und der Abstand **wächst** mit der Seitentiefe. Traversiert man die **ganze** Tabelle Seite für Seite (im Dashboard), summiert sich der Offset-Nachteil deutlich auf.

### Code-Walkthrough

- `service/ReadService.java` → `pageByOffset(...)` / `pageByKeyset(...)`: dieselbe Projektion, nur die WHERE-/OFFSET-Klausel unterscheidet sich.
- `controller/ReadController.java` → `/r2/offset` und `/r2/keyset`.
- Client ( `stages.ts`, `traverseOffset` / `traverseKeyset` ): blättert die ganze Tabelle durch und misst die Gesamtzeit. Keyset merkt sich die **letzte id** als nächsten `afterId`.

### Fallstricke & Trade-offs

- Keyset kann nicht direkt auf Seite  $n$  springen** — es kennt nur „weiter ab hier“. Für „Springe zu Seite 500“ ist Offset nötig (oder eine andere Strategie). Das ist der zentrale Trade-off: Keyset ist schneller, aber weniger flexibel.
- Keyset braucht eine **eindeutige, indizierte Sortierspalte** (hier `id`). Bei Sortierung nach nicht-eindeutigen Spalten muss man den Schlüssel zusammensetzen (z. B. `(ts, id)`).
- Offset ist einfacher und für **flache** Seiten (die ersten paar) völlig in Ordnung.

### Ausblick

- R3** löst ein anderes Problem: nicht *wie viel*, sondern *wann* die Daten ankommen — Streaming senkt die Time-To-First-Byte.

## R3 — Server-seitiges Streaming (NDJSON)

Track: Read · Vorgänger: R2 · Nachfolger: R4

### Konzept

R0/R1 bauen die **komplette** Antwort im Speicher zusammen und senden sie erst, wenn sie fertig ist. Zwei Nachteile: Der Speicher wächst mit der Datenmenge, und der Client sieht **lange gar nichts** (hohe Time-To-First-Byte).

R3 **streamt**: Der Server liest die Zeilen über einen **DB-Cursor** blockweise und schreibt jede Zeile sofort als **NDJSON** (ein JSON-Objekt pro Zeile) in den Ausgabestrom. So liegt immer nur ein kleiner Block im Speicher, und das erste Byte erreicht den Client fast sofort.

Der Cursor entsteht durch drei Zutaten (PostgreSQL-JDBC): `autoCommit = false` + `fetchSize > 0` + `TYPE_FORWARD_ONLY`.

### Was die Messung zeigt

100 000 Zeilen:

| Stufe                 | TTFB (erstes Byte) | Gesamtzeit |
|-----------------------|--------------------|------------|
| R1 (buffered)         | ~0,454 s           | ~0,80 s    |
| <b>R3 (streaming)</b> | <b>~0,043 s</b>    | ~0,85 s    |

Die **TTFB sinkt um ~10×** — der Client kann sofort anfangen zu verarbeiten/anzuzeigen, während der Rest noch fließt. Die Gesamtzeit ist ähnlich; der Gewinn liegt in **Reaktivität und konstantem Speicher**, nicht in weniger Bytes.

### Code-Walkthrough

- `service/ReadService.java` → `streamNdjson(OutputStream)`: öffnet eine Connection mit `autoCommit=false`, setzt `fetchSize=1000`, iteriert das `ResultSet` und schreibt je Zeile `objectMapper.writeValueAsBytes(dto) + \n`.
- `controller/ReadController.java` → `r3Stream()`: gibt eine `StreamingResponseBody` zurück (`produces = "application/x-ndjson"`), die Spring MVC streamend abarbeitet.
- Client (`stages.ts`, `r3Stage`): das Harness misst über den Stream-Reader die **TTFB**; die Zeilen werden per Zeilenumbruch gezählt.

Hinweis: Weil die Antwort früh „committed“ ist, kann der `ServerTimingFilter` seinen Header nicht mehr setzen — bei R3 zählt daher die clientseitige TTFB/Gesamtzeit.

### Fallstricke & Trade-offs

- Transaktion offen halten**: Der Cursor lebt in einer offenen Transaktion, solange gestreamt wird. Bei sehr langsamen Clients bindet das eine DB-Verbindung.
- Fehler mitten im Stream**: Ist der HTTP-Status (200) schon gesendet, kann man ihn nicht mehr ändern — Fehlerbehandlung während des Streamens ist kniffliger.
- NDJSON ist ideal fürs zeilenweise Verarbeiten; ein einzelnes großes JSON-Array wäre für Streaming unpraktisch (Klammern/Kommas).

### Ausblick

- R5** spart die Arbeit ganz — per Caching für wiederholte Abfragen.

- **R8** treibt das Streaming ins **Reaktive** (R2DBC) und zeigt Server-Sent Events.

## R4 — HTTP-Kompression (gzip)

Track: Read · verwandt mit: R1 · Nachfolger: R5

### Konzept

R4 überträgt **dieselben** Projektionsdaten wie R1, aber **gzip-komprimiert**. JSON komprimiert hervorragend, weil es sehr repetitiv ist (immer dieselben Feldnamen, ähnliche Werte). Der Browser dekomprimiert transparent ( `Content-Encoding: gzip` ).

Damit R1 (unkomprimiert) und R4 (komprimiert) **direkt vergleichbar** sind, komprimiert der Endpoint bewusst **manuell** — statt globaler Servlet-Kompression, die alle Antworten beträfe.

### Was die Messung zeigt

100 000 Zeilen, `payloadLength=64` :

| Stufe                         | Nutzlast auf der Leitung | Faktor ggü. R0 |
|-------------------------------|--------------------------|----------------|
| R0 (volle Entity)             | 35,4 MB                  | —              |
| R1 (Projektion)               | 10,7 MB                  | ~3,3×          |
| <b>R4 (Projektion + gzip)</b> | <b>2,4 MB</b>            | <b>~15×</b>    |

gzip drückt die schon schlanke Projektion nochmals um ~4,5×. Von R0 zu R4 sind es zusammen rund **15× weniger Bytes**. Im Dashboard zeigt die Spalte „**Wire KB**“ genau diese echte Leitungsgröße (aus dem Header `X-Wire-Bytes` ).

### Warum ein eigener Header für die Wire-Größe?

Der Browser **dekomprimiert** die Antwort, bevor unser Mess-Code den Body liest — wir würden also die *unkomprimierte* Größe zählen und die Ersparnis nicht sehen. Deshalb schickt der Server die tatsächliche komprimierte Größe im Header `X-Wire-Bytes` mit, und das Harness nutzt sie für „Wire KB“ und den MB/s-Durchsatz.

### Code-Walkthrough

- `controller/ReadController.java` → `r4Compressed()` : serialisiert die Projektion zu JSON ( `ObjectMapper` — in Boot 4 aus dem Paket `tools.jackson.databind` ), komprimiert sie mit `GZIPOutputStream` , setzt `Content-Encoding: gzip` und `X-Wire-Bytes` und gibt die Bytes zurück.
- `harness.ts` → `measure()` : liest `X-Wire-Bytes` und füllt damit `wireBytes` .

### Fallstricke & Trade-offs

- CPU vs. Bandbreite:** Kompression kostet Rechenzeit auf beiden Seiten. Bei sehr kleinen Antworten oder CPU-Engpässen lohnt sie sich nicht — die globale Servlet-Kompression hat deshalb eine Mindestgröße.
- Nicht doppelt komprimieren:** Schon komprimierte Inhalte (Bilder, Protobuf) profitieren kaum; gzip darüber ist verschwendete CPU.
- Für maximale Ersparnis wäre `br` (Brotli) oft noch besser als gzip.

### Ausblick

- R3** senkt statt der Bytes die **Time-To-First-Byte** durch Streaming.
- R5** vermeidet die Arbeit ganz — per Caching für wiederholte Abfragen.

## R5 — Caching (Caffeine)

Track: Read · Vorgänger: R4 · Nachfolger: R6

### Konzept

Die schnellste Abfrage ist die, die man **gar nicht** ausführt. R5 nutzt die **Spring-Cache-Abstraktion** mit **Caffeine** (In-Memory-Cache): Das Ergebnis einer teuren Aggregat-Abfrage (Statistik je Kategorie über alle Zeilen) wird nach dem ersten Aufruf gecacht. Jeder weitere Aufruf wird **aus dem Speicher** bedient — die Datenbank sieht ihn nie.

```
@Cacheable("categoryStats")
public List<CategoryStat> categoryStats() { ... } // nur beim ersten Mal wird die DB befragt
```

### Was die Messung zeigt

100 000 Zeilen; der R5-Lauf leert erst den Cache, misst dann **kalt** und **warm**:

| Aufruf               | Zeit     |
|----------------------|----------|
| kalt (aus der DB)    | ~0,111 s |
| warm (aus dem Cache) | ~0,012 s |

Der Cache-Treffer ist hier ~9× schneller — und bei teureren Abfragen (mehr Daten, komplexere Aggregation) wird der Unterschied noch größer.

### Code-Walkthrough

- `config/CacheConfig.java` : `@EnableCaching` (ohne das werden die Annotationen ignoriert).
- `service/ReadService.java` → `categoryStats()` mit `@Cacheable("categoryStats")` und `evictCategoryStats()` mit `@CacheEvict`.
- `controller/ReadController.java` → `GET /r5` (gecached) und `POST /r5/evict` (Cache leeren).
- Client ( `stages.ts` , `r5Stage` ): leert erst den Cache, misst kalt und warm und zeigt beide Werte in der Notiz; die Serverms-Spalte zeigt den warmen (Cache-)Wert.

### Fallstricke & Trade-offs

- **Invalidierung ist das schwere Problem**: Ändern sich die Daten, liefert der Cache veraltete Ergebnisse, bis er geleert wird — genau das zeigt der Test (R5 bleibt stabil, bis `evict`). Time-to-live, ereignisbasiertes Evict oder Versionierung sind die üblichen Strategien.
- **Speicher**: Der Cache belegt RAM; Größe/Ablauf müssen begrenzt werden (Caffeine kann das). Für verteilte Systeme braucht man einen gemeinsamen Cache (z. B. Redis).
- Caching lohnt sich für **wiederholte, teure, selten geänderte** Abfragen — nicht für alles.

### Ausblick

- **R6** beschleunigt nicht durch Vermeiden, sondern durch **Parallelisieren** unabhängiger Abfragen.

## R6 — Parallel-Queries (Virtual Threads)

Track: Read · Vorgänger: R5 · Nachfolger: R7

### Konzept

Ein „Dashboard“ besteht oft aus **mehreren unabhängigen** Abfragen (Kennzahl A, Kennzahl B, ...). Führt man sie **nacheinander** aus, summiert sich ihre Zeit. Sind sie voneinander unabhängig, kann man sie **gleichzeitig** ausführen — dann zählt nur die *langsamste*.

R6 führt 8 unabhängige Aggregat-Abfragen aus, wahlweise seriell oder parallel. Parallel bekommt jede Abfrage einen eigenen **Virtual Thread**.

### Was die Messung zeigt

100 000 Zeilen, 8 Abfragen:

| Modus           | Serverzeit     |
|-----------------|----------------|
| seriell         | ~241 ms        |
| <b>parallel</b> | <b>~150 ms</b> |

Hier bringt Parallelität ~1,6×. **Warum nicht 8×?** Die Abfragen sind **CPU-gebunden** (Full-Scans über die im Speicher liegende Tabelle) und konkurrieren um die verfügbaren CPU-Kerne. Der Gewinn ist also durch die Kernzahl begrenzt.

Wichtige Nuance: Der Effekt ist viel größer, wenn die Abfragen **I/O-/Wartezeit-lastig** sind (langsame Platte, entfernte Dienste, Locks) — dann warten die Threads, statt Kerne zu belegen, und Parallelität zahlt sich stark aus. Genau dafür sind Virtual Threads gemacht.

### Voraussetzung: genug Verbindungen

Wie bei W7 gilt: Parallele DB-Abfragen brauchen **mehrere Verbindungen**. Der HikariCP-Pool muss groß genug sein (hier  $10 \geq 8$  parallele Abfragen), sonst serialisieren sich die Abfragen am Pool.

### Code-Walkthrough

- `service/ReadService.java` → `dashboard(boolean parallel)` : führt die 8 Abfragen seriell in einer Schleife oder parallel über `Executors.newVirtualThreadPerTaskExecutor()` aus.
- `controller/ReadController.java` → `GET /r6?parallel=...` : misst die Serverzeit und gibt sie im `BenchmarkResult`-Envelope zurück.
- Client (`stages.ts`, `r6SeqStage` / `r6ParStage`): zwei Stufen für den direkten Vergleich.

### Fallstricke & Trade-offs

- Nur für unabhängige Abfragen.** Hängt B von A ab, bringt Parallelität nichts.
- Mehr ist nicht immer besser:** Zu viele gleichzeitige Abfragen erzeugen Contention (CPU, Pool, Locks). Der optimale Grad ist ein Tuning-Thema.
- Fehlerbehandlung wird komplexer (mehrere Tasks, Teil-Ergebnisse).

## Ausblick

- **R7** senkt die Serialisierungskosten und die Bytes durch ein **Binärformat** statt JSON.

## R7 — Binärformat (CBOR) statt JSON

Track: Read · Vorgänger: R6 · Nachfolger: R8

### Konzept

JSON ist textbasiert und dadurch etwas „geschwätzig“: Feldnamen als Text, Zahlen als Ziffernfolgen, viele Trennzeichen. **CBOR** (Concise Binary Object Representation, RFC 8949) ist ein **binäres** Format mit demselben Datenmodell wie JSON — nur kompakter und schneller zu (de)serialisieren.

R7 liefert dieselben Projektionsdaten wie R1, aber als CBOR. Damit der Client das Binärformat nicht selbst dekodieren muss, kommt die Zeilenzahl im Header `X-Rows`.

### Was die Messung zeigt

100 000 Zeilen (ohne payload):

| Stufe            | Nutzlast       | Faktor ggü. R1                 |
|------------------|----------------|--------------------------------|
| R1 (JSON)        | 10,75 MB       | —                              |
| <b>R7 (CBOR)</b> | <b>8,06 MB</b> | <b>~0,75× (≈ 25 % kleiner)</b> |

CBOR spart hier ~25 %. Wie viel genau, hängt von den Daten ab: Bei vielen wiederholten **Feldnamen** und kompakten **Zahlen** ist der Gewinn größer; bei langen Freitexten kleiner.

### Code-Walkthrough

- `controller/ReadController.java` → `r7Cbor()` : serialisiert die Projektion mit einem `CBORMapper` (Jackson 3, Paket `tools.jackson.dataformat.cbor`) und setzt `Content-Type: application/cbor`, `X-Wire-Bytes` und `X-Rows`.
- Client (`stages.ts`, `r7Stage`): misst die Byte-Größe direkt und liest die Zeilenzahl aus `X-Rows`.

### Fallstricke & Trade-offs

- Nicht menschenlesbar:** CBOR lässt sich nicht einfach im Browser/Log ansehen — schlechter für Debugging.
- Client-Support nötig:** Der Empfänger braucht einen CBOR-Decoder. JSON ist universell.
- Kompression relativiert den Vorteil:** gzip über JSON (siehe R4) holt oft mehr heraus als CBOR unkomprimiert. Am kleinsten wird es mit **CBOR + Kompression**.
- Für maximale Kompaktheit/Schema-Sicherheit sind schemabasierte Formate (Protobuf, Avro) noch effizienter — aber aufwändiger (Schema-Pflege, Codegen).

### Ausblick

- R8** schließt den Read-Track mit dem **reaktiven** Ansatz (R2DBC) ab.

## R8 — Reaktives Streaming über R2DBC

Track: Read · Vorgänger: R7 · Abschluss des Read-Tracks

### Konzept

R8 liest die Daten **reaktiv** über **R2DBC**: Die Query liefert einen Reactor- `Flux`, den Spring MVC elementweise als NDJSON streamt. Es ist das Read-Gegenstück zu W8 und die reaktive Variante von R3.

```
public Flux<MeasurementDto> streamReactive() {
    return Flux.usingWhen(connectionFactory.create(),
        conn -> Flux.from(conn.createStatement(PROJECTION_SQL).fetchSize(1000).execute())
            .flatMap(result -> result.map((row, meta) -> new MeasurementDto(...))),
        conn -> conn.close());
}
```

### Was die Messung zeigt — die ehrliche Lektion

100 000 Zeilen:

| Stufe                             | TTFB          | Gesamtzeit   |
|-----------------------------------|---------------|--------------|
| R3 (Servlet-Streaming)            | ~0,02 s       | ~1,0 s       |
| <b>R8 (R2DBC reaktiv, in MVC)</b> | <b>~1,0 s</b> | <b>~14 s</b> |

**R8 ist hier deutlich langsamer als R3** — nicht schneller. Das ist kein Bug, sondern eine wichtige Einsicht:

Einen reaktiven `Flux` in einen **blockierenden MVC-Endpoint** zu brücken kostet viel: Die Reactive→MVC-NDJSON-Brücke serialisiert und flusht Element für Element, dazu kommt der Overhead des reaktiven Row-Mappings. Der eigentliche Nutzen von R2DBC/Reactive entsteht erst in einer **vollständigen WebFlux-Anwendung** und unter **hoher Nebenläufigkeit** (viele gleichzeitige, teils langsame Clients) — dann skaliert das nicht-blockierende Modell mit wenigen Threads. Für ein einzelnes Bulk-Streaming ist der klassische Servlet-Cursor (R3) klar überlegen.

Damit spannt sich der Bogen zu W8: **Reaktiv ist ein anderes Programmiermodell für Nebenläufigkeit/IO — kein Turbo-Knopf für Durchsatz.**

### Code-Walkthrough

- `common/R2dbcSupport.java`: leitet die R2DBC- `ConnectionFactory` aus der JDBC-`DataSource` ab (dieselbe Utility nutzt auch W8). Bewusst **keine Spring-Bean** — sonst schaltet Boots `DataSourceAutoConfiguration` die JDBC-`DataSource` ab ( `@ConditionalOnMissingBean(ConnectionFactory)` ).
- `service/ReadService.java` → `streamReactive()`: der reaktive Lese-Flux.
- `controller/ReadController.java` → `r8Reactive()`: gibt den `Flux` zurück; Spring MVC streamt ihn als NDJSON ( `produces = "application/x-ndjson"` ).

### Fallstricke & Trade-offs

- Mischbetrieb MVC + Reactive** ist selten optimal — entweder ganz blockierend (mit Virtual Threads sehr effizient!) oder ganz reaktiv (WebFlux).
- Backpressure** funktioniert nur, wenn die ganze Kette sie respektiert; die MVC-Brücke ist hier der limitierende Faktor.

- R2DBC hat ein eigenes, kleineres Ökosystem als JDBC (kein Hibernate, andere APIs).

## Fazit des Read-Tracks

Von R0 (alles, volle Entities) bis R7 (schlank + binär) haben wir die Nutzlast um **Größenordnungen** verkleinert (Projektion → Kompression → Binärformat), die **TTFB** durch Streaming (R3) drastisch gesenkt, mit **Keyset-Pagination** (R2) tiefe Seiten konstant schnell gemacht, mit **Caching** (R5) Arbeit ganz vermieden und mit **Parallelität** (R6) unabhängige Abfragen beschleunigt. R8 zeigt schließlich: Das reaktive Modell ist mächtig, aber kontextabhängig. Die Kernbotschaft des ganzen Projekts: **Es gibt kein „schneller ist immer besser“ — jede Technik hat ihren Kontext, ihre Trade-offs und ihren richtigen Ort.**

# Aufgaben für Studenten

---

Diese Aufgaben führen durch das Projekt — vom Beobachten über das Verstehen bis zum Erweitern. Sie sind in drei Stufen gegliedert:

-  **Einsteiger** — beobachten, messen, vorhersagen
-  **Fortgeschritten** — Parameter verändern, Ursachen erklären
-  **Profi** — Code erweitern, eigene Stufen bauen

Arbeite immer nach dem Muster **Vorhersagen** → **Messen** → **Erklären**. Schreibe deine Vorhersage *auf*, bevor du misst — der Aha-Effekt entsteht bei den Überraschungen.

## 0. Vorbereitung

1. Starte Datenbank, Server und Client (siehe README).
2. Seede einen Datenbestand: **100 000 Zeilen, Payload-Länge 32**.
3. Mach dich mit dem Dashboard vertraut: Was bedeuten die Spalten *Server ms*, *TTFB*, *Wire KB*, *Zeilen/s*, *MB/s*?

## 1. Write-Track

### 1.1 Vorhersagen

Bevor du misst: Ordne die Stufen W0–W6 nach erwartetem Durchsatz (langsamste zuerst). Führe dann ► **Alle Write** aus und vergleiche mit deiner Reihenfolge.

### 1.2 Der überraschende Sprung

- Um welchen Faktor ist **W2** schneller als **W1**? Warum bringt das bloße Weglassen des HTTP-Overheads (W0 → W1) so wenig, aber die gemeinsame Transaktion (W1 → W2) so viel?
- Stichwort: Was passiert bei einem `COMMIT` physisch auf der Platte?

### 1.3 Warum rohes JDBC ab W3?

- Lies `docs/W3.md`. Warum kann **Hibernate** die INSERTs bei `GenerationType.IDENTITY` nicht als Batch bündeln? Was müsste man an der Entity ändern, damit es ginge — und welchen Preis hätte das?

### 1.4 Warum ist COPY (W6) so viel schneller?

- Erkläre in zwei Sätzen, was `COPY ... FROM STDIN` anders macht als viele `INSERT` s.

## 2. Read-Track

### 2.1 Nutzlast schrumpfen

Führe R0, R1, R4, R7 aus (Wire KB vergleichen).

- Welchen Anteil spart **Projektion** (R0 → R1), welchen **Kompression** (R1 → R4)?
- Ist **CBOR** (R7) kleiner oder größer als **gzip-JSON** (R4)? Warum?

### 2.2 Zeit vs. Bytes

- R1 und R3 übertragen ähnlich viele Bytes. Warum ist die **TTFB** von R3 trotzdem ~10× kleiner?

- Für welche Art von Anwendung ist niedrige TTFB wichtiger als Gesamtzeit?

## 2.3 Pagination ●

- Führe R2 (Offset) und R2 (Keyset) aus. Warum wächst der Offset-Nachteil mit der Tiefe?
- **Denkaufgabe:** Nenne einen Anwendungsfall, in dem Offset-Pagination *nötig* ist und Keyset nicht genügt.

## 2.4 Caching ●

- Führe R5 aus. Erkläre die Notiz „kalt X ms → warm Y ms“.
  - **Falle:** Was liefert R5 zurück, wenn sich die Daten ändern, der Cache aber nicht geleert wird? Verifiziere im Code ( `ReadService.categoryStats` + Test `r5CachesUntilEvict` ).
- 

## 3. Experimente (Parameter verändern) ●

Miss jeweils **vorher/nachher** und notiere den Effekt.

### 3.1 Batch-Größe

In `WriteService.BATCH_SIZE` (Default 1000) ändern auf 100, dann 5000. Baue neu ( `ci 21` bzw. `ci 21` ), miss W3/W4. Wo liegt das Optimum, und warum ist „größer“ nicht immer besser?

### 3.2 Payload-Länge

Seede mit Payload-Länge 0, dann 256. Wie verändert sich der Unterschied zwischen **R0** und **R1** (Projektion)? Warum?

### 3.3 Connection-Pool ●

Setze die Pool-Größe klein: starte den Server mit `HIKARI_MAX_POOL=2` ( `HIKARI_MAX_POOL=2 java -jar ...` ). Führe **W7** (parallel) aus.

- Was passiert mit dem Durchsatz? Erkläre den Zusammenhang Pool-Größe ↔ Parallelität.

### 3.4 Lasttest ●

Führe `infra/load/read-load.js` mit k6 aus (siehe `infra/load/README.md` ). Erhöhe die VU-Zahl (20 → 50 → 100). Ab wann steigt die **p95-Latenz** stark? Vergleiche mit der Pool-Größe.

---

## 4. Erweitern (Code schreiben) ●

### 4.1 Neue Read-Stufe: Filter-Projektion

Baue eine Stufe **R1b**, die nur Zeilen einer bestimmten `category` zurückgibt.

- Server: neue Methode in `ReadService` + Endpoint in `ReadController` .
- Client: neue `Stage` in `stages.ts` , in `STAGES` registrieren.
- Test: Integrationstest analog `ReadStagesIntegrationTest` .
- **Definition of Done:** Endpoint funktioniert · Test grün · im Dashboard sichtbar.

### 4.2 Neue Write-Stufe: UPSERT

Baue eine Stufe, die per `INSERT ... ON CONFLICT` schreibt (idempotenter Ingest). Überlege: Funktioniert `rewriteBatchedInserts` damit noch? (Tipp: Doku W4 lesen.)

### 4.3 Metrik ergänzen

Erweitere das Dashboard um eine Spalte „Bytes/Zeile“. Wo im Client ( `harness.ts` / `ResultsChart.tsx` ) musst du ansetzen?

---

## 5. Quiz (zum Abschluss)

1. Nenne die **zwei größten Hebel** im Write-Track und begründe die Reihenfolge.
  2. Warum ist **reaktiv** (W8/R8) beim Bulk-Insert *nicht* schneller — und wann *ist* es die richtige Wahl?
  3. Eine API liefert 50 MB JSON pro Request und ist „zu langsam“. Nenne **drei** verschiedene Techniken aus diesem Projekt, um das zu verbessern, mit je einem Satz Trade-off.
  4. Richtig oder falsch: „Ein größerer Connection-Pool ist immer besser.“ Begründe.
  5. Was misst der Header `Server-Timing`, und warum reicht er bei der Streaming-Stufe R3 nicht aus?
- 

## 6. Abgabe / Reflexion

Schreibe eine halbe Seite zur Leitfrage des Projekts:

„Warum gibt es kein *schneller ist immer besser?*“

Belege deine Antwort mit **zwei** konkreten Messungen aus diesem Projekt, bei denen die „fortgeschrittenere“ Technik *nicht* die schnellere war — und erkläre den Kontext, in dem sie trotzdem die richtige Wahl wäre.

---

Musterlösungen und Referenzzahlen: siehe die Stufen-Dokumente `docs/W0-W8.md` und `docs/R0-R8.md` sowie die Integrationstests unter `server/src/test/`.